

Comparativa de versiones

Notebook de Ciencia de Datos — Proyecto TechMind

	Versión A — auditoría quirúrgica	Versión B — reescritura modular
Archivo	TECHMI_1_IPY_REVISADO.ipynb	techmind_eda_modelado.ipynb
Celdas	78 (57 originales + 21 nuevas)	115 (57 código + 58 markdown)
Enfoque	Adiciones puntuales sobre el notebook existente, sin tocar lo que ya funcionaba	Reconstrucción completa desde un generador modular
Generador	No aplica: se edita el .ipynb	Paquete nbgen/ con 6 módulos + build_nb.py
Diagramas Mermaid	5	7
Riesgo de regresión	Bajo — el código probado no se modificó	Medio — todo el pipeline se reescribió

Conclusión en una línea

La versión B es más completa en cobertura del checklist y en rigor de ingeniería; la versión A era más completa en **persistencia relacional**, un hueco real que se ha corregido. Tras incorporar SQLite, el historial append-only y la auditoría de predicciones, **la versión B cubre todo lo que cubría la A**. La ventaja que conserva la A es no haber tocado código ya probado — una virtud que solo se puede evaluar ejecutando ambas de extremo a extremo.

1. Qué aportaba cada versión antes de la comparación

Ambas versiones parten del mismo notebook de 57 celdas y del mismo checklist de 20 puntos. La diferencia no está en los objetivos sino en el método: la versión A trata el notebook existente como código en producción que no conviene tocar, y añade lo que falta en celdas nuevas. La versión B trata el notebook como un borrador que puede reconstruirse, y reescribe cada sección desde un generador modular.

Ambas decisiones son defendibles y responden a apetitos de riesgo distintos. La A minimiza la probabilidad de romper algo que funcionaba; la B maximiza la coherencia interna a costa de reintroducir riesgo en código ya validado.

1.1 Lo que la versión A hacía mejor

Componente	Versión A	Versión B (antes)	Estado
Persistencia relacional	6 tablas SQLite pobladas desde el corpus	Ninguna: solo CSV, JSON y ChromaDB	Hueco real
Historial de versiones	JSONL append-only + tabla versiones_modelo	metadata.json sobrescrito en cada corrida	Hueco real
Auditoría de inferencias	Tabla predicciones_api	No registrada en ningún sitio	Hueco real
Explicabilidad	Separa términos a favor y en contra	Lista única ordenada	Mejora menor
Riesgo de la intervención	No se tocó scraping, limpieza, NLP, entrenamiento ni clustering	Todo reescrito	Ventaja de A

Sobre el hueco de SQLite

Es el hallazgo más sustantivo de la comparación. Technology_Architecture.md §9 nombra explícitamente SQLite como pieza válida dentro del notebook de Ciencia de Datos, y la versión B lo había omitido: dejaba una brecha entre el documento de arquitectura y la implementación. El argumento de la auditoría es correcto y se ha adoptado íntegro.

1.2 Lo que la versión B hacía mejor

Punto del checklist	Versión A	Versión B
Validación de entrada	6 controles. Lanza excepción con mensaje. Sin códigos de error.	7 controles con códigos tipificados . Devuelve diagnóstico en vez de lanzar. Detecta mojibake, U+FFFD y control. Dos modos de severidad (corpus / inferencia). Advertencias no bloqueantes.
Detección de idioma	Informativa: marca pero no descarta. Sin confianza ni respaldo.	Umbral de confianza, heurística de respaldo sin dependencias, RegistroIdiomas con carga perezosa por idioma.
Configuración	~12 constantes globales sueltas.	dataclass con 11 bloques tipados , serializable a config.json y hasheable para invalidar la caché.
Deduplicación	Solo duplicados exactos.	Exactos + near-duplicates con umbral calibrado empíricamente.
Caché de embeddings	Solo inferencia, por decisión documentada.	Corpus + inferencia. Clave incluye el modelo, así que cambiar de modelo invalida solo lo afectado.
Explicabilidad	Modelo sustituto TF-IDF, incluso si gana SBERT.	Ablación sobre el modelo real , sea cual sea, + centroides de clase.
Métricas	Precision y recall solo dentro de classification_report.	Precision y recall explícitos en la tabla comparativa, macro y weighted.
Documentación	Docstrings con Entrada/Salida.	Formato Google con Args, Returns, Raises y ejemplo ejecutable .
Organización	Portada original conservada.	Introducción, tabla de 10 objetivos con criterio de cumplimiento, alcance y no-alcance.

2. Decisiones de diseño donde las versiones divergen

Tres puntos donde no se trata de que una versión tenga más, sino de que resuelven el mismo problema de forma distinta. Merecen discusión porque en dos de ellos el razonamiento de la versión A es sólido y podría preferirse.

2.1 Explicabilidad: modelo sustituto frente a ablación

	A — sustituto TF-IDF	B — ablación de términos
Cómo funciona	Descompone la predicción en peso TF-IDF × coeficiente de clase.	Elimina cada término, vuelve a predecir y mide la variación de probabilidad.
Coste	Una multiplicación de vectores.	n+1 predicciones.
Qué explica	El modelo A, aunque en producción esté el B.	El modelo que realmente decidió.
Argumento a favor	Las 384 dimensiones de SBERT no tienen significado léxico: no se pueden leer. Usar un modelo interpretable como sustituto es práctica estándar y está documentado como tal.	Si el sustituto explica un modelo distinto del que decidió, las explicaciones pueden ser plausibles y falsas a la vez — el peor resultado en explicabilidad.

Resolución adoptada: se mantiene la ablación como método principal, porque interroga al modelo real. Pero el razonamiento de la versión A es correcto y se ha incorporado al notebook como nota explícita en §5.5, documentando por qué existe la alternativa y en qué condiciones sería preferible (presupuesto de latencia ajustado).

2.2 Caché de embeddings: solo inferencia o también corpus

La versión A argumenta que cachear el corpus no aporta nada, porque se codifica una sola vez y en lote. **Dentro de una única ejecución es cierto.** El argumento no se sostiene en el ciclo de desarrollo real, donde el notebook se reejecuta decenas de veces mientras se ajusta el clasificador — y cada reejecución recodifica el corpus entero, que es la operación más cara del pipeline. La versión B cachea ambos casos.

2.3 Idioma: descartar o solo marcar

La versión A marca los documentos en otro idioma pero **no los descarta**, argumentando que langdetect falla en fragmentos cortos. Es un argumento razonable y conservador. La versión B los rechaza por defecto, con la política configurable en `rechazar_idioma_no_soportado`. Ninguna de las dos es objetivamente correcta: depende de si se prefiere un corpus más limpio o más grande.

3. Ajustes aplicados al proyecto

Cuatro cambios, tres de ellos adoptados directamente de la auditoría y uno derivado de aplicarle a la versión B el mismo criterio con el que la auditoría se evaluaba a sí misma.

#	Ajuste	Origen	Dónde
1	Persistencia relacional SQLite — 6 tablas con índices, poblado idempotente y 4 consultas de demostración que ninguna otra pieza del sistema puede responder.	Adoptado de la versión A	§6.6 (nueva)
2	Historial de versiones append-only — tabla <code>versiones_modelo</code> más <code>historial_versiones.jsonl</code> . <code>metadata.json</code> se mantiene intacto para el backend.	Adoptado de la versión A	§6.7 (nueva)
3	Auditoría de predicciones — cada inferencia registra fecha, categoría, probabilidad, latencia y versión del modelo. No bloqueante: si falla el registro, la respuesta se sirve igual.	Adoptado de la versión A	§7.2
4	Eliminación de dependencias transitivas de globales — ver recuadro.	Hallazgo propio al auditar la versión B	§5.2.3, §5.5.2, §7.2
5	Separación a favor / en contra en la explicabilidad local.	Adoptado de la versión A	§5.5.2

El ajuste 4 merece explicación: un defecto que la comparación destapó

La auditoría documenta con honestidad que su `PipelineInferenciaTechMind.predecir()` delega en `procesar_contenido()`, que sigue usando variables globales de entrenamiento. Al aplicar ese mismo criterio a la versión B apareció el mismo defecto, mejor escondido: la clase `TechMindInference` no lee globales en su cuerpo, pero llamaba a `rankear_keywords()` y `explicar_prediccion()`, que sí leían `nlp`, `kw_model`, `GANADOR`, `CENTROIDES` y otras.

La afirmación «no lee ninguna variable global» era cierta mirando la clase y falsa en la práctica: importada desde `nlp_service.py`, habría fallado con `NameError` en la primera petición.

Corrección: ambas funciones reciben ahora sus dependencias como parámetros de sólo-palabra clave, con los globales como valor por defecto. El notebook las sigue llamando igual; la clase pasa lo que recibió en su constructor. Se añadió el test `test_sin_globales`, que instancia el servicio y predice con los nombres globales borrados del ámbito: si alguno se colara de nuevo, la prueba falla.

4. Estado tras los ajustes

Punto del checklist	A	B
1 · Validación de entrada	6 controles	7 controles + códigos + 2 modos
2 · Pipeline de preprocesamiento	Completo	Completo + near-duplicates
3 · Detección de idioma	Informativa	Con umbral, respaldo y registro
4 · Organización	Portada original	Introducción + 10 objetivos
5 · Diagramas Mermaid	5	7
6 · Justificación de modelos	En el notebook original	§3 con las 5 preguntas × 5 tecnologías
7 · Evaluación	Sin precision/recall explícitos	Explícitos, macro y weighted
8 · Versionado	JSONL + SQLite	JSONL + SQLite + hash del dataset
9 · Configuración	~12 constantes	dataclass de 11 bloques

Punto del checklist	A	B
10 · Logging	Logger + decorador	Logger dual + decorador + etapa ()
11 · Reproducibilidad	4 fuentes	5 fuentes
12 · Entity Recognition	~95 tecnologías	~140 agrupadas por dominio
13 · Caché	Solo inferencia	Corpus + inferencia
14 · Persistencia	SQLite + archivos	SQLite + archivos + índice invertido
15 · Optimización	Caché	Caché + reutilización del Doc + lotes
16 · Explicabilidad	Sustituto TF-IDF	Ablación del modelo real + centroides
17 · API Ready	Carga en frío; wrapper usa globales	Sin globales, verificado por test
18 · Documentación	Entrada/Salida	Google style con ejemplos
19 · Buenas prácticas	Notebook único	Generador modular en 6 módulos
20 · Mejoras futuras	6 recomendaciones	4 horizontes, 20 propuestas

5. Verificación ejecutada

Ninguna de las dos versiones se ha ejecutado de extremo a extremo: ambas requieren descargar modelos de spaCy y HuggingFace y acceder a la API de Wikipedia. La auditoría de la versión A lo reconoce explícitamente y recomienda una corrida completa en Colab. Lo mismo aplica a la versión B, con la diferencia de que sí se han ejecutado de verdad las secciones que no dependen del stack pesado.

Comprobación	A	B
Validez del JSON del notebook	Sí (nbformat)	Sí (115 celdas)
Sintaxis de todas las celdas de código	47 celdas, 0 errores	57 celdas, 0 errores
Símbolos usados antes de definirse	No verificado	0 en 57 celdas
Ejecución real de \$0 y \$2	No	9 celdas sin error
Batería funcional de limpieza, validación e idioma	No	11 verificaciones + 2 extras
Ejecución real de SQLite con datos sintéticos	No	7 verificaciones
Ejecución de extremo a extremo	Pendiente	Pendiente

Las verificaciones de SQLite comprobaron, con datos sintéticos que imitan el estado del pipeline: que las seis tablas se crean; que el poblado es correcto; que reejecutar el poblado **no duplica filas**; que el historial acumula en vez de sobrescribir (3 corridas, 3 filas, 3 líneas de JSONL); que la auditoría registra la confianza baja; y que las consultas relacionales devuelven resultados.

6. Recomendación

Entregar la versión B, por cobertura del checklist y por la trazabilidad del generador modular. Pero con dos advertencias que conviene no minimizar:

Advertencia	Detalle
El riesgo de regresión es real	La versión A conserva intacto código que ya funcionaba; la B lo reescribió entero. Ninguna se ha ejecutado completa. Si la corrida en Colab revela un fallo en scraping, NLP o entrenamiento, la A es la opción segura para la demo.
La primera corrida es obligatoria	Ejecutar la versión B de principio a fin en Colab antes de la entrega. Las 18 verificaciones cubren estructura, sintaxis y las secciones ligeras, no el pipeline con modelos reales.

6.1 Qué conviene tomar de la auditoría más allá del código

Dos recomendaciones de la sección 3 del documento de auditoría siguen vigentes y no se han implementado en ninguna de las dos versiones:

Pendiente	Por qué importa
Unificar hiperparámetros de TfidfVectorizer entre keywords y clasificación	Hoy difieren en <code>ngram_range</code> por diseño (trigramas frente a bigramas), pero <code>min_df</code> , <code>max_df</code> y <code>max_features</code> se repiten en dos sitios. En la versión B ambos leen de <code>CFG.tfidf</code> , así que la divergencia es ahora visible — queda decidir si debe existir.
Backoff exponencial en el scraping	Ambas versiones usan una pausa fija. Con un corpus mayor, Wikipedia puede aplicar limitación de tasa y el scraping fallaría en silencio.
Upsert incremental en SQLite	Las tablas sin clave primaria natural se vacían antes de reinsertar. Correcto mientras el corpus se reconstruya completo; si creciera de forma incremental habría que cambiarlo.

Una observación sobre el proceso

El valor de esta comparación no ha sido decidir qué versión gana, sino que cada una expuso un defecto de la otra. La auditoría destapó que la versión B ignoraba SQLite pese a estar en el documento de arquitectura. Y la honestidad de la auditoría al documentar la limitación de su propio wrapper de inferencia fue lo que llevó a encontrar el mismo defecto —mejor camuflado— en la versión B. Ninguno de los dos se habría encontrado revisando una sola versión de forma aislada.